

Claude Code za 90 dana — besplatni kurs samostalnog učenja

Nauči da koristiš Claude Code maksimalno — od prve komande do sopstvenih agenata i automatizacije. 80% prakse, 20% teorije. Bez sertifikata, sa nečim boljim: 6 završenih mini-projekata.

Kome je namenjen: svakome ko zna osnove rada u terminalu (otvoriti terminal, `cd` u folder) i želi da nauči da programira uz AI agenta. Ne moraš biti profesionalni programer — Claude piše kod, ti učiš da ga vodiš.

Preduslovi:

- Instaliran **Claude Code** (CLI ili desktop aplikacija) i Claude nalog
- Instaliran **Git** (treba tek od Modula 6, ali instaliraj odmah)
- Bilo koji operativni sistem (Windows / Mac / Linux)

Kako se kurs koristi:

- 12 nedelja, 6 modula (svaki modul = 2 nedelje)
- Ritam: 5 dana × 45–60 minuta vežbe + 1 dan retrospektive (šta sam naučio, šta mi nije jasno)
- Svaki modul gradi **mini-projekat od nule** — ne treba ti nikakav postojeći kod
- Vežbe su označene sa **V** (npr. V1.3 = Modul 1, vežba 3). Za svaku vežbu postoji rešenje na kraju kursa u odeljku **Rešenja zadataka** — ali **prvo pokušaj sam**. Greška je deo učenja; rešenje gledaj tek kad zapneš.
- Na kraju svakog modula je **Provera znanja** — zadatak koji radiš bez uputstava, samo sa ciljem. To je tvoj "ispit" umesto sertifikata.

Pregled modula:

Modul	Tema	Nedelje	Mini-projekat
1	Claude CLI osnove	1–2	Dnevnik troškova (Python konzolna aplikacija)
2	CLAUDE.md i memorija	3–4	Lični portfolio sajt (HTML/CSS/JS)
3	Plan mode i Loop	5–6	Kviz aplikacija (web)
4	MCP serveri	7–8	Prognoza + beleške (spoljni podaci + baza)
5	Paralelni subagenti	9–10	Analiza open-source projekta
6	Git integracija i automatizacija	11–12	Završni projekat po izboru

Modul 1 — Claude CLI osnove (nedelje 1–2)

Mini-projekat: "Dnevnik troškova" — konzolna aplikacija u Python-u koja beleži i sabira lične troškove. Kreće se od potpuno praznog foldera.

ZAŠTO ovo postoji

Claude Code nije chat. Chat ti da tekst koji ti onda ručno kopiraš u fajlove. Claude Code je **agent**: sam čita tvoje fajlove, sam ih menja, sam pokreće komande u terminalu, vidi grešku koju je izazvao i sam je ispravi. Tvoj posao se pomera sa "kucam kod" na "opisujem šta hoću i proveravam rezultat". Ceo kurs je zapravo učenje te nove veštine: **kako precizno reći agentu šta hoćeš i kako proveriti da je to zaista uradio**.

Teorija modula (pročitaj jednom, vraćaj se po potrebi)

Dva režima rada:

- **Interaktivni:** ukucaš `claude` u folderu projekta → otvara se razgovor koji traje dok ga ne zatvoriš. Ovde radiš 95% vremena.
- **Jednokratni:** `claude -p "pitanje ili zadatak"` → Claude odradi, ispiše rezultat i završi. Za skriptovanje i brze provere (detaljnije u Modulu 6).

Osnovne komande unutar razgovora:

Komanda	Šta radi	Zašto ti treba
<code>/help</code>	spisak svih komandi	prva stvar koju kucaš u novom alatu
<code>/clear</code>	briše ceo razgovor	novi zadatak = čist kontekst = bolji rezultati
<code>/compact</code>	sažima razgovor da oslobodi prostor	dugačke sesije pune "kontekst" — Claude ima ograničenu radnu memoriju
<code>/model</code>	menja model (brži/jeftiniji vs. pametniji)	ne treba ti najjači model za sitnice
<code>/resume</code>	vraća se na raniju sesiju	nastavi sutra gde si stao
<code>/rewind</code>	vraća razgovor (i po želji fajlove) na raniju tačku	"poništi" kad Claude ode u pogrešnom smeru
<code>Esc</code>	prekida Claude-a usred rada	vidiš da radi pogrešno? Ne čekaj kraj

Referenciranje fajlova: ukucaj `@` pa ime fajla (npr. `@troskovi.py`) — Claude će taj fajl sigurno pročitati. Bez toga sam odlučuje šta će čitati (obično pogodi, ali `@` je garancija).

Permission modes (režimi dozvola) — Claude te pita za dozvolu pre nego što nešto izmeni ili pokrene. ZAŠTO: agent koji sam menja fajlove i pokreće komande mora imati kočnicu. Režimi (menjaju se sa **Shift+Tab**):

- **default** — pita za svaku izmenu/komandu. Počni ovde: vidiš tačno šta radi.
- **acceptEdits** — izmene fajlova prolaze automatski, komande i dalje pita. Kad stekneš poverenje.
- **plan** — Claude sme samo da ČITA i pravi plan, ništa ne menja (Modul 3).
- **bypassPermissions** — ne pita ništa. Samo za izolovana/bezbedna okruženja — preskoči za sada.

Nedelja 1 — prvi koraci i prva aplikacija

Dan 1 — instalacija i prvi razgovor

1. Napravi prazan folder `dnevnik-troskova` i uđi u njega u terminalu.
2. Pokreni `claude`. Ukucaj `/help` i pročitaj šta postoji (ne moraš ništa zapamtiti).
3. **V1.1:** Zatraži od Claude-a da napravi CSV fajl sa 10 izmišljenih troškova (datum, opis, kategorija, iznos) i Python skript `troskovi.py` koji čita taj CSV i ispisuje ukupan zbir. Pokreni skript. [Rešenje](#)
4. Obrati pažnju: Claude te je pitao za dozvolu pre pisanja fajla i pre pokretanja komande. Pročitaj ŠTA te tačno pita pre nego što odobriš — ta navika je zlata vredna.

Dan 2 — dogradnja: unos novih troškova

1. Nastavi u istom folderu (`claude`, pa ako želiš stari razgovor: `/resume`).
2. **V1.2:** Traži da skript dobije meni: opcija za unos novog troška (koji se dopisuje u CSV) i opcija za prikaz zbira. Testiraj oba. [Rešenje](#)
3. Isprobaj `@troskovi.py` u poruci — vidi kako Claude reaguje kad mu eksplicitno daš fajl.

Dan 3 — mesečni izveštaj

1. **V1.3:** Traži izveštaj po mesecima i po kategorijama (npr. "hrana: 4.500 din, prevoz: 1.200 din"). Neka Claude sam predloži kako da izgleda ispis, pa ga doteruj: "poravnaj kolone", "dodaj ukupno na dnu". [Rešenje](#)
2. Poenta vežbe: **ne moraš unapred znati tačan izgled — doteruj kroz razgovor.** To je normalan način rada.

Dan 4 — namerno izazivanje greške (debug)

1. **V1.4:** Ručno otvori CSV u editoru i pokvari jedan red (npr. obriši iznos ili upiši slovo umesto broja). Pokreni skript — pucaće. Onda u Claude ukucaj samo: "pokreni troskovi.py i ispravi grešku". Posmatraj: pročitao grešku, naći uzrok, ispraviti i ponovo pokrenuti da proveriti. [Rešenje](#)
2. ZAŠTO je ovo ključno: ovo je petlja koju ćeš koristiti stalno — **Claude ume sam da proveriti svoj rad ako mu daš način da ga proveriti** (pokretanje, test, primer).

Dan 5 — jednokratni režim

1. Izadi iz razgovora. **V1.5:** Iz terminala pokreni: `claude -p "koliko redova ima troskovi.csv i koji je najveći pojedinačni trošak?"` [Rešenje](#)
2. Isprobaj još 2–3 jednokratna pitanja o svom kodu. Primeti razliku: nema razgovora, dobiješ odgovor i kraj.

Dan 6 — retrospektiva: Zapiši (u svesku ili fajl `BELESKE.md`): šta me je iznenadilo? Kada je Claude pogrešio i zašto — da li je moj opis bio neprecizan? Koju komandu još nisam probao?

Nedelja 2 — sesije, kontekst i režimi dozvola

Dan 1 — nastavljjanje sesija

1. Zatvori terminal skroz. Otvori ponovo, uđi u folder, pokreni `claude --resume` (ili `claude` pa `/resume`) i izaberi jučerašnju sesiju.
2. **V1.6:** Nastavi TAČNO gde si stao: traži da se doda kategorija "ostalo" za troškove bez kategorije. Primeti da se Claude "seća" svega iz te sesije. [Rešenje](#)

Dan 2 — kontekst i /compact

1. Teorija u 3 rečenice: Claude ima ograničenu "radnu memoriju" (kontekst). Sve što ste pričali i svi fajlovi koje je čitao je pune. Kad se napuni, stariji delovi se sažimaju — zato dugačke sesije postaju "zaboravne".
2. Praksa: pogledaj indikator konteksta (donji deo ekrana). Ukucaj `/compact` i vidi šta se desi. Pravilo za poneti: **novi zadatak** → `/clear`. Ne vuci jedan razgovor danima.

Dan 3 — /rewind i ispravljanje smeru

1. **V1.7:** Traži nešto namerno maglovito, npr. "sredi malo izgled ispisa". Kad Claude uradi nešto što ti se ne sviđa, upotrebi `/rewind` da se vratiš, pa ponovi zahtev ali precizno ("koristi tabelu sa kolonama datum/opis/iznos, iznos poravnat udesno"). Uporedi rezultate. **Rešenje**
2. Lekcija: kvalitet rezultata = kvalitet opisa. `/rewind` + bolji opis je često brže od 5 rundi ispravki.

Dan 4 — permission modes u praksi

1. **V1.8:** Prebaci se u **acceptEdits** (Shift+Tab) i traži neku manju doradu (npr. lepši datum u ispisu). Primeti da izmene fajlova više ne traže potvrdu, ali komande i dalje traže. Vрати se na default. **Rešenje**
2. Razmisli i zapiši: u kojim situacijama bi acceptEdits bio opasan? (npr. folder sa važnim fajlovima koji nema git)

Dan 5 — PROVERA ZNANJA Modula 1 V1.9 (bez uputstava): U novoj sesiji (`/clear`), koristeći sve naučeno, dodaj u aplikaciju izvoz mesečnog izveštaja u JSON fajl. Uslovi: (1) moraš bar jednom upotrebiti `@` referencu, (2) moraš pustiti Claude-a da sam pokrene i proverí kod, (3) na kraju jednokratnom komandom `claude -p` proverí da JSON fajl valja. **Rešenje i kriterijumi uspeha**

Dan 6 — retrospektiva.

Modul 2 — CLAUDE.md i memorija (nedelje 3–4)

Mini-projekat: "Lični portfolio sajt" — statički sajt (HTML/CSS/JS) sa početnom stranom, stranicom o projektima i kontakt formom. Od praznog foldera.

ZAŠTO ovo postoji

Claude svaku novu sesiju kreće **prazan** — ne zna ništa o tvom projektu, tvojim pravilima, tvom stilu.

`CLAUDE.md` je fajl koji Claude automatski pročita na početku SVAKE sesije u tom folderu. To je razlika između saradnika kome svaki dan iznova objašnjavaš sve — i saradnika koji je pročitao "kućni red" pre nego što je seo za sto.

Teorija modula

Tri nivoa `CLAUDE.md` fajla:

Fajl	Važi za	Primer sadržaja
<code>~/ .claude/CLAUDE.md</code> (globalni)	SVE tvoje projekte	"odgovaraj mi na srpskom", "commit poruke piši kratko"
<code>CLAUDE.md</code> u folderu projekta	taj projekat, i sve koji rade na njemu (ide u git)	pravila projekta, konvencije, poznate zamke
<code>CLAUDE.local.md</code>	taj projekat, samo tvoja mašina (ne ide u git)	tvoje lične prečice, lokalne putanje

Šta SPADA u CLAUDE.md: pravila koja se ne vide iz koda. Konvencije ("sav tekst na sajtu je na srpskom"), zabrane ("ne koristi framework, čist HTML/CSS/JS"), zamke ("fajl X ne diraj, generisan je automatski"), komande projekta ("sajt se testira otvaranjem index.html").

Šta NE SPADA: ono što Claude može sam da vidi iz koda (spisak fajlova, šta koja funkcija radi) — to zastareva i puni kontekst bez potrebe.

Prečica #: ukucaj `#` na početku poruke (npr. `# uvek koristi css varijable za boje`) — Claude to trajno zapiše u memoriju/CLAUDE.md umesto tebe.

`/init` : komanda koja analizira postojeći projekat i sama sastavi početni CLAUDE.md.

Nedelja 3 — bol bez CLAUDE.md, pa lek

Dan 1 — sajt bez pravila (namerno)

- Novi folder `portfolio`. **V2.1:** Traži početnu stranu: ime, kratak opis, 3 sekcije (O meni / Projekti / Kontakt). NE zadaj nikakva pravila o stilu/jeziku. Samo prihvati šta ponudi. **Rešenje**

Dan 2 — "zaboravnost" na delu

- Zatvori sesiju. Otvori NOVU (bez `--resume`!). **V2.2:** Traži novu stranicu `projekti.html` sa 3 kartice projekata. Uporedi je sa početnom: boje? font? jezik teksta? struktura? Velika je verovatnoća da nešto odudara — Claude nije imao odakle da zna tvoje konvencije. Zapiši šta konkretno odudara. **Rešenje**

Dan 3 — pišemo CLAUDE.md

1. **V2.3:** Napravi `CLAUDE.md` u folderu projekta (možeš i tražiti od Claude-a da ga napravi po tvom diktatu) sa bar 5 pravila, npr: jezik svog teksta, paleta boja (konkretne hex vrednosti), bez framework-a i biblioteka, svaka stranica koristi zajednički `style.css`, imenovanje fajlova malim slovima. **Rešenje sa primerom kompletnog fajla**

Dan 4 — test leka

1. Nova sesija (ne resume). **V2.4:** Traži treću stranicu (`kontakt.html` sa formom). NE pominji nijedno pravilo. Proveri: da li je poštovao `CLAUDE.md` bez podsećanja? Ako neko pravilo nije ispoštovao — pravilo je verovatno nejasno napisano; doteraj formulaciju. **Rešenje**

Dan 5 — prečica # i /init

1. **V2.5:** Usred rada ukucaj `#` pravilo koje ti je falilo (npr. `# dugmad uvek imaju hover efekat`). Pogledaj gde ga je upisao. Zatim u nekom DRUGOM postojećem folderu (bilo koji stari projekat ili preuzet repo) pokreni `/init` i pročitaj šta je generisao — šta bi ti izbacio/dodao? **Rešenje**

Dan 6 — retrospektiva.

Nedelja 4 — hijerarhija i održavanje memorije

Dan 1 — globalni `CLAUDE.md`

1. **V2.6:** Napravi/dopuni globalni `~/.claude/CLAUDE.md` (na Windowsu: `C:\Users\<ti>\.claude\CLAUDE.md`) sa 2–3 LIČNA pravila koja želiš svuda (jezik komunikacije, stil commit poruka...). Otvori sesiju u portfolio projektu i proveri da važe OBA fajla — globalni i projektni. **Rešenje**

Dan 2 — šta ne spada u `CLAUDE.md`

1. Vežba razmišljanja: dodaj u `CLAUDE.md` namerno lošu stavku — detaljan opis šta svaki fajl radi. Sledeće sesije primeti da to (a) zastareva čim se kod promeni, (b) troši kontekst. Obriši je. Pravilo: **`CLAUDE.md` sadrži ono što se NE VIDI iz koda.**

Dan 3–4 — dorada sajta uz živi `CLAUDE.md`

1. Dodaj sajtu šta god želiš (galerija, tamna tema, animacije). Svaki put kad Claude-a moraš da PODSETIŠ na nešto — to je kandidat za `CLAUDE.md`. Dopunjavaj ga usput (`#` prečica).

Dan 5 — PROVERA ZNANJA Modula 2 V2.7 (bez uputstava): Simuliraj "novog člana tima": obriši sve sesije iz glave — otvori potpuno novu sesiju i traži: "dodaj stranicu sa cenovnikom usluga". Kriterijum uspeha: nova stranica se uklapa u sajt (jezik, boje, struktura, stil) BEZ ijednog tvog podsećanja. Ako ne — `CLAUDE.md` ima rupu; nađi je i zakrpi. **Rešenje i kriterijumi**

Dan 6 — retrospektiva.

Modul 3 — Plan mode i Loop (nedelje 5–6)

Mini-projekat: "Kviz aplikacija" — web kviz: pitanja sa 4 ponuđena odgovora, merenje vremena, rezultat na kraju i tabela rekorda (localStorage). Od praznog foldera.

ZAŠTO ovo postoji

Kod malih izmena Claude može odmah da krene — ako pogreši, ispravka je jeftina. Kod VEĆIH poduhvata (nova aplikacija, refaktorisanje, izmena koja dira 10 fajlova) pogrešan smer znači sat vremena čišćenja. **Plan mode** obrće redosled: Claude prvo sme samo da ČITA i istražuje, pa ti predloži plan, i tek kad plan odobriš — sme da menja. Pregledati plan od 20 redova je nemerljivo jeftinije od pregledanja 500 redova pogrešnog koda.

Teorija modula

- U plan mode se ulazi sa **Shift+Tab** (dok ne vidiš "plan mode"), ili pokretanjem `claude` pa prebacivanjem pre prvog zahteva.
- U planu Claude istražuje kod, postavlja pitanja i na kraju predloži plan. Ti možeš: **odobriti**, **tražiti izmene** (samo napiši šta da promeni — plan se dorađuje), ili **odustati**.
- Kada plan mode NE treba: sitnice ("promeni boju dugmeta"). Precenjena ceremonija za male stvari usporava.
- **Extended thinking:** za teške probleme dodaj u zahtev "think hard" / "razmisli dobro" — Claude dobija više "vremena za razmišljanje" pre odgovora. Koristi za: dizajn arhitekture, čudne bagove, odluke sa više opcija.
- `/loop` : ponavlja zadatak u intervalima (npr. proveru nešto svakih 5 minuta). Za nadgledanje i ponavljajuće poslove.

Nedelja 5 — kviz kroz plan mode

Dan 1 — plan za strukturu

1. Novi folder `kviz`. Uđi u plan mode. **V3.1:** Traži: "napravi web kviz aplikaciju: pitanja sa 4 odgovora, 10 pitanja opšte kulture, rezultat na kraju". PROČITAJ plan pažljivo pre odobravanja: da li pominje fajlove koje očekuješ? Da li je nešto prekomplikovano? **Rešenje**
2. Odobri plan i pusti ga da implementira. Otvori kviz u browseru.

Dan 2 — namerno odbijanje plana

1. Plan mode. **V3.2:** Traži tabelu rekorda (ime igrača + rezultat + datum, čuvanje u localStorage, prikaz top 10). Kad stigne plan — NE odobravaj odmah: traži bar 2 izmene (npr. "rekordi da se pamte i po kategoriji" ili "dodaj dugme za brisanje rekorda"). Gledaj kako se plan dorađuje pa tek onda odobri. **Rešenje**
2. Lekcija: plan je PREDLOG, ne obaveza. Najveća vrednost plan mode-a je pregovaranje PRE koda.

Dan 3 — merenje vremena + think hard

1. **V3.3:** Traži tajmer po pitanju (npr. 15 sekundi, isteklo vreme = netačan odgovor) i bonus poene za brzinu. U zahtev dodaj "razmisli dobro o ivicama: šta ako korisnik promeni tab, šta ako odgovori u poslednjoj sekundi". Uporedi kvalitet plana sa običnim. **Rešenje**

Dan 4 — kada plan mode NE treba

1. Bez plan mode-a, direktno: 3–4 sitne izmene ("veći font pitanja", "zeleno/crveno bojenje odgovora", "dodaj još 5 pitanja"). Oseti razliku u tempu. Zapiši svoje pravilo palca: od koje veličine zadatka TEBI vredi plan mode?

Dan 5 — /loop

1. **V3.4:** Isprobaj `/loop` na bezopasnom zadatku, npr: "svakih 5 minuta proveri da li u folderu ima novih .html fajlova bez odgovarajućeg linka u index.html i javi mi". Pusti 2–3 kruga pa zaustavi. Razmisli: šta bi u TVOM stvarnom radu vredelo ovako nadgledati? [Rešenje](#)

Dan 6 — retrospektiva.

Nedelja 6 — veliko refaktorisanje kroz plan

Dan 1–2 — priprema i plan refaktorisanja

1. Tvoj kviz je verovatno u 1–2 fajla. **V3.5:** U plan mode traži refaktorisanje u module: odvojeni fajlovi za pitanja (podaci), logiku kviza, tajmer, rekorde i prikaz. Uslov u zahtevu: "posle refaktorisanja sve mora raditi identično kao pre". Pregledaj plan DETALJNO — refaktorisanje je najlakše mesto da se nešto usput pokvari. [Rešenje](#)
2. Odobri i pusti. Testiraj SVE funkcije kviza posle (odigraj celu partiju, proveri rekorde).

Dan 3–4 — dorade na novoj strukturi

1. Dodaj kategorije pitanja i izbor kategorije na početku. Primeti: da li je izmena LAKŠA na modularnoj strukturi? To je bio smisao refaktorisanja.

Dan 5 — PROVERA ZNANJA Modula 3 V3.6 (bez uputstava): Kroz plan mode, od plana do implementacije: dodaj kvizu režim "2 igrača naizmenično" sa uporednim rezultatom na kraju. Kriterijumi: plan si bar jednom dorađivao pre odobravanja; posle implementacije si lično testirao oba režima (1 i 2 igrača). [Rešenje i kriterijumi](#)

Dan 6 — retrospektiva.

Modul 4 — MCP serveri (nedelje 7–8)

Mini-projekat: "Prognoza + beleške" — aplikacija koja prikazuje vremensku prognozu (spoljni podaci) i tvoje beleške po danima (lokalna SQLite baza).

ZAŠTO ovo postoji

Claude iz kutije "vidi" samo dve stvari: tvoje fajlove i tvoj terminal. Ne vidi tvoju bazu, tvoj browser, tvoj Slack, tvoj GitHub nalog. **MCP (Model Context Protocol)** je standardan način da mu daš nova "čula" — MCP server je mali program koji Claude-u izlaže alate ("pročitaj tabelu", "otvori stranicu u browseru", "pošalji poruku"). Umesto da ti prepričavaš Claude-u šta piše u bazi, on je pogleda sam.

Teorija modula

Komande:

- `claude mcp add <ime> ...` — dodaje server (tačna sintaksa zavisi od servera; svaki server u svojoj dokumentaciji ima gotovu komandu za kopiranje)
- `claude mcp list` — spisak povezanih servera i njihovo stanje
- `claude mcp remove <ime>` — uklanjanje
- `/mcp` unutar sesije — status servera i prijava (za servere koji traže nalog)

Scope (domet) — gde se konfiguracija pamti:

Scope	Važi za	Kada
<code>local</code> (default)	samo ti, samo taj projekat	probanje, lični alati
<code>project</code>	svi na projektu (fajl <code>.mcp.json</code> ide u git)	tim deli iste alate
<code>user</code>	ti, u svim projektima	alati koje svuda koristiš

Bezbednosna napomena (bitno!): MCP server ima pristup onome što mu daš. Dodaješ samo servere kojima veruješ (zvanični, poznati). Server koji čita bazu — razmisli da li mu treba i pisanje.

Nedelja 7 — prvi MCP: baza podataka

Dan 1 — aplikacija (bez MCP-a, za početak)

- Novi folder `prognoza-beleske`. **V4.1:** Traži aplikaciju (može Python + malo web-a, neka Claude predloži): SQLite baza `beleske.db` sa tabelom beležaka (datum, tekst), unos i pregled beležaka po danu. Ubaci 5–6 probnih beležaka. **Rešenje**

Dan 2 — SQLite MCP server

- V4.2:** Pronađi i dodaj MCP server za SQLite (traži od samog Claude-a: "koji je preporučen način da dodam MCP server za sqlite i daj mi tačnu komandu za moju bazu `beleske.db`"). Posle dodavanja: `claude mcp list` mora da ga pokaže. **Rešenje**
- Test-pitanje Claude-u: "koliko beležaka ima u bazi i koja je najnovija?" — mora odgovoriti čitanjem PREKO MCP-a, ne pisanjem skripta.

Dan 3 — osećaj razlike

1. **V4.3:** Isti zadatak uradi na oba načina i uporedi: (a) pitaj Claude-a nešto o podacima DOK je MCP aktivan; (b) ukloni server (`claude mcp remove`), pa postavi isto pitanje — sada će morati da piše skript/komandu da bi došao do podataka. Zapiši razliku u brzini i broju koraka. Vрати server. **Rešenje**

Dan 4 — spoljni podaci (prognoza)

1. **V4.4:** Dodaj u aplikaciju prikaz vremenske prognoze za tvoj grad (postoje besplatni API-ji bez registracije — pusti Claude-a da predloži i implementira poziv IZ KODA aplikacije). **Rešenje**
2. Diskusija sa samim sobom (zapiši): prognozu vuče KOD aplikacije preko API-ja, a bazu čita CLAUDE preko MCP-a. U čemu je razlika? (Odgovor u rešenju V4.7.)

Dan 5 — retrospektiva + sređivanje.

Nedelja 8 — browser MCP i registar

Dan 1–2 — browser MCP

1. **V4.5:** Poveži browser MCP (npr. "Claude in Chrome" ekstenzija ili Playwright MCP — pitaj Claude-a šta je dostupno za tvoje okruženje i kako se instalira). Zadatak: neka Claude SAM otvori tvoju aplikaciju u browseru, unese novu belešku kroz formu i proveri da se pojavila na spisku. **Rešenje**
2. ZAŠTO je ovo veliko: do sada si TI bio Claude-ove oči za sve što se dešava u browseru. Sad može sam da klikće i čita stranicu — može sam da testira ono što napravi.

Dan 3 — registar MCP servera

1. **V4.6:** Istraži šta postoji: pitaj Claude-a za poznate/zvanične MCP servere, ili pretraži "MCP servers registry" u browseru. Izaberi JEDAN koji tebi lično deluje korisno (GitHub? fajl sistem? nešto treće?), dodaj ga i smisli mini-zadatak da ga isprobaš. **Rešenje**

Dan 4 — .mcp.json za deljenje

1. Dodaj jedan od svojih servera sa `--scope project` i pogledaj nastali `.mcp.json` fajl u projektu. To je fajl koji bi kolega dobio kroz git i imao iste alate kao ti.

Dan 5 — PROVERA ZNANJA Modula 4 V4.7 (bez uputstava, teorijska + praktična): (1) Objasni svojim rečima, u 5 rečenica, razliku između MCP servera i API poziva iz koda aplikacije — i navedi po jedan primer kada je koji pravi izbor. (2) Praktično: neka Claude, koristeći browser MCP + SQLite MCP zajedno, testira celu aplikaciju i napiše ti kratak izveštaj šta radi a šta ne. **Rešenje i kriterijumi**

Dan 6 — retrospektiva.

Modul 5 — Paralelni subagenti (nedelje 9–10)

Mini-projekat: analiza tuđeg open-source projekta + custom agent "recenzent" koji pregleda tvoj kviz iz Modula 3.

ZAŠTO ovo postoji

Dva problema koje subagenti rešavaju:

1. **Kontekst je ograničen.** Ako Claude mora da pročita 50 fajlova da bi odgovorio na jedno pitanje, tih 50 fajlova zauvek puni tvoju sesiju. Subagent je "izviđač": ode, pročita sve u SVOM odvojenom kontekstu, i vrati ti samo zaključak od 10 redova.
2. **Neki poslovi su nezavisni.** Tri pitanja o tri različita dela koda ne moraju da čekaju jedno drugo — tri subagenta rade paralelno.

Kada se NE isplate: mali zadaci. Subagent kreće "od nule" (ne vidi vaš razgovor), pa za sitnicu potroši više na upoznavanje nego na posao.

Teorija modula

- Subagente pokrećeš prirodnim jezikom: "pošalji subagenta da istraži X", "pokreni 3 agenta paralelno: prvi neka..., drugi..., treći...".
- Ugrađeni tipovi: **Explore** (samo čita — bezbedan izviđač), **Plan** (dizajnira pristup), **general-purpose** (sve može).
- **Custom agenti** — sopstveni specijalisti: fajl u `.claude/agents/` sa opisom uloge, uputstvima i (opciono) ograničenim alatima. Prave se i komandom `/agents`. ZAŠTO: umesto da svaki put kucaš isti dugačak prompt "pregledaj kod i proverih ovih 10 stvari", to jednom zapišeš kao agenta.

Nedelja 9 — izviđači na tuđem kodu

Dan 1 — kloniranje mete

1. Izaberi poznat open-source projekat srednje veličine i kloniraj ga, npr: `git clone https://github.com/pallets/flask` (ili bilo koji projekat koji te zanima). Uđi u folder, pokreni `claude`.

Dan 2 — jedan izviđač

1. **V5.1:** Pošalji jednog Explore agenta: "pošalji subagenta da istraži kako je u ovom projektu organizovano rukovanje greškama i da mi vrati kratak izveštaj sa putanjama ključnih fajlova". Primeti: tvoja sesija je ostala "lagana" — agent je čitao u svom kontekstu. **Rešenje**

Dan 3 — tri paralelna izviđača

1. **V5.2:** Jedan zahtev, tri agenta paralelno, tri različita pitanja o projektu (npr: kako se pišu testovi / kako je organizovana konfiguracija / koje spoljne biblioteke koristi i za šta). Traži od Claude-a da ti na kraju spoji nalaze u jedan pregled. **Rešenje**

Dan 4 — poređenje sa/bez

1. **V5.3:** Isti tip pitanja (novo pitanje o projektu) postavi jednom DIREKTNO (bez subagenta, neka sam čita fajlove u tvojoj sesiji) i jednom PREKO subagenta. Uporedi: potrošnju konteksta (indikator), vreme,

kvalitet odgovora. Zapiši kada bi koji pristup birao. [Rešenje](#)

Dan 5 — retrospektiva.

Nedelja 10 — sopstveni specijalista

Dan 1–2 — custom agent "recenzent"

1. **V5.4:** Napravi custom agenta u `.claude/agents/recenzent.md` (možeš kroz `/agents` ili tražiti od Claude-a da ga napravi). Uloga: pregleda kod po checklist-i — čitljivost, imena promenljivih, ponavljanje koda, očigledni bagovi, komentari tamo gde su nužni. Neka bude READ-ONLY (samo čita, ništa ne menja) i neka izveštaj vraća kao listu nalaza sa putanjom i predlogom. [Rešenje sa kompletnim primerom fajla](#)

Dan 3 — recenzent na delu

1. **V5.5:** Vрати se u folder kviza iz Modula 3 (prekopiraj agenta ili ga napravi kao user-level). Pusti recenzenta na ceo kviz. Pročitaj nalaze KRITIČKI: sa čim se slažeš, sa čim ne? Ispravi 2–3 nalaza (može i tako što ispravke tražiš od običnog Claude-a). [Rešenje](#)

Dan 4 — doterivanje agenta

1. Recenzent je sigurno prijavio i nešto nebitno. Doteraj njegova uputstva (npr. "ne prijavljuj stilske sitnice ako nisu dogovorene u CLAUDE.md") i pusti ga ponovo. ZAŠTO: agent se kalibriše kroz iteracije, kao i saradnik.

Dan 5 — PROVERA ZNANJA Modula 5 V5.6 (bez uputstava): Na open-source projektu po izboru: (1) tri paralelna Explore agenta sa tri pitanja koje SAM smisliš, (2) spojen izveštaj, (3) tvoja procena u 5 rečenica: šta bi u ovom projektu bilo najteže menjati i zašto — na osnovu izveštaja. Kriterijum: izveštaj sadrži konkretne putanje fajlova, a tvoja procena se poziva na nalaze. [Rešenje i kriterijumi](#)

Dan 6 — retrospektiva.

Modul 6 — Git integracija i automatizacija (nedelje 11–12)

Završni projekat: kompletna mala aplikacija po TVOM izboru — od ideje do GitHub repozitorijuma, primenom svega naučenog.

ZAŠTO ovo postoji

Do sada si pravio kod. Ovaj modul je o tome kako se kod ŽIVI u stvarnom radu: verzije (git), pregled izmena pre spajanja (review — jeftinije je naći bug u pregledu nego u produkciji), i automatizacija (hooks, sopstvene komande, headless režim) — da se ponavljajući deo tvog rada spakuje i izvršava sam.

Teorija modula

- **Git kroz Claude:** Claude ume ceo git tok — `git init`, grane, smislene commit poruke (sam pregleda izmene i sastavi poruku), a uz **GitHub CLI** (`gh`) i PR-ove. Ti i dalje ODOBRAVAŠ svaku komandu.
- `/code-review` — pregleda tvoje trenutne izmene i prijavi bagove/probleme pre nego što ih commit-uješ.
- **Hooks** — automatske akcije koje se dešavaju na događaje (npr. "posle svake izmene fajla pokreni testove"). Definišu se u `.claude/settings.json`. VAŽNO ZAŠTO: hook izvršava sam alat (garantovano, svaki put), a ne Claude po svom nahođenju — pravila koja MORAJU da se dese idu u hook, ne u `CLAUDE.md`.
- **Custom slash komande** — tvoj workflow kao komanda: fajl u `.claude/commands/` (npr. `objavi.md`) postaje `/objavi`. Umesto da svaki put kucaš isti niz zahteva.
- **Headless režim** — `claude -p "zadatak"` iz skripte/CI-ja: Claude kao alat u automatizaciji, bez razgovora.

Nedelja 11 — git tok od nule

Dan 1 — repo i prvi commit

1. Izaberi ideju za završni projekat (predlozi: lista za kupovinu sa deljenjem, evidencija treninga, čitač RSS vesti, planer obaveza — ili nešto svoje). Novi folder.
2. **V6.1:** Odmah na startu: traži od Claude-a `git init`, `.gitignore` po tipu projekta i `CLAUDE.md` (Modul 2 znanje!). Zatim prva funkcionalnost i prvi commit — ali PROČITAJ commit poruku koju je predložio pre nego što odobriš. **Rešenje**

Dan 2 — grana + funkcionalnost + review

1. **V6.2:** Nova funkcionalnost kompletno na grani: "napravi granu za funkcionalnost X, implementiraj je" → pa PRE commit-a pokreni `/code-review`. Pročitaj nalaze, ispravi opravdane, pa commit. **Rešenje**

Dan 3 — GitHub i PR

1. Napravi (besplatan) GitHub nalog ako nemaš i instaliraj `gh` CLI (`gh auth login`).
2. **V6.3:** Traži od Claude-a da objavi repo na GitHub i otvori Pull Request sa tvoje grane, sa opisom izmena. Pogledaj PR u browseru — to je format u kome timovi razmenjuju kod. Spoji ga (merge). **Rešenje**

Dan 4 — hook

1. **V6.4:** Podesi hook koji se pokreće posle svake Claude-ove izmene fajla — za početak nešto prosto i vidljivo (npr. za Python projekat: provera sintakse izmenjenog fajla). Testiraj: zatraži izmenu i uveri se da se hook okinuo. [Rešenje sa primerom settings.json](#)
2. Razmisli: koje bi pravilo iz tvog CLAUDE.md zapravo trebalo da bude hook? (Podsetnik: CLAUDE.md = molba, hook = garancija.)

Dan 5 — retrospektiva.

Nedelja 12 — sopstvena komanda, headless i ZAVRŠNI ISPIT

Dan 1 — custom slash komanda

1. **V6.5:** Napravi `.claude/commands/objavi.md` — komandu `/objavi` koja: pokrene proveru koda, sastavi commit poruku iz izmena, commit-uje i push-uje. Isprobaj je na stvarnoj izmeni. [Rešenje sa kompletnim primerom fajla](#)

Dan 2 — headless

1. **V6.6:** Napiši (uz Claude-ovu pomoć) skript koji koristi `claude -p` da svakodnevno može da odgovori na pitanje o projektu, npr: "izlistaj TODO komentare u kodu i predloži kojim redom da se rešavaju". Pokreni ga ručno i pogledaj izlaz. [Rešenje](#)

Dan 3–5 — ZAVRŠNI ISPIT V6.7: Završi svoj projekat primenom SVEGA — checklist:

- ☐ CLAUDE.md postoji od starta i ažuriran je (Modul 2)
- ☐ Bar jedna veća funkcionalnost urađena kroz plan mode (Modul 3)
- ☐ Bar jedan MCP server u upotrebi (Modul 4)
- ☐ Bar jedno istraživanje kroz subagenta (Modul 5)
- ☐ Ceo git tok: grane, review pre commit-a, PR na GitHub (Modul 6)
- ☐ Bar jedan hook ili custom komanda u projektu (Modul 6)
- ☐ Aplikacija RADI — lično si je testirao od početka do kraja

Kriterijumi uspeha

Dan 6 — završna retrospektiva: šta od naučenog ostaje u tvom svakodnevnom radu? Napiši sopstveni "top 5 pravila za rad sa Claude Code".

Šta posle kursa

- **Zvanična dokumentacija:** <https://code.claude.com/docs> — posebno delovi o hooks, skills i SDK
 - **Anthropic-ovi tekstovi o best practices** za agentic coding (pretraži "Claude Code best practices")
 - Unutar samog alata: `/help` i pitanja tipa "kako da u Claude Code uradim X" — alat ume da objasni sebe
 - Ideje za nastavak: sopstveni skill-ovi (`.claude/skills/`), Claude Agent SDK (pravljenje sopstvenih agenata), naprednije MCP integracije
-
-

REŠENJA ZADATAKA

Pravilo: rešenje gledaš tek posle bar 15 minuta sopstvenog pokušaja. Prompt-ovi ispod su REFERENTNI — tvoja formulacija ne mora biti ista, bitno je da sadrži iste ključne informacije. Ako tvoj rezultat radi a drugačiji je od rešenja — to je uspeh, ne greška.

Modul 1 — rešenja

Rešenje V1.1

Referentni prompt:

Napravi CSV fajl `troskovi.csv` sa kolonama `datum`, `opis`, `kategorija`, `iznos` i 10 izmišljenih redova (datumi iz poslednja 2 meseca, kategorije: hrana, prevoz, računi, zabava). Zatim napravi Python skript `troskovi.py` koji čita taj CSV i ispisuje ukupan zbir svih iznosa. Pokreni ga da proveriš da radi.

Očekivani ishod: dva fajla u folderu; Claude je sam pokrenuo skript i pokazao ti ispis zbira. **Česte greške:**

- *Prompt bez strukture kolona* ("napravi mi neki CSV sa troškovima") — dobiješ nasumične kolone koje kasnije moraš prepravljati. Što ranije fiksiraš strukturu podataka, manje prepravki kasnije.
- *Nisi tražio da pokrene skript* — Claude tada može predati kod koji nije proveren. Navika: "...i pokreni da proveriš" na kraju zahteva.

Rešenje V1.2

Referentni prompt:

Dodaj u `troskovi.py` meni sa opcijama: 1) unos novog troška (pita za datum, opis, kategoriju, iznos i dopiše red u CSV), 2) prikaz ukupnog zbira, 3) izlaz. Posle izmene pokreni i testiraj unos jednog troška.

Očekivani ishod: meni radi; novi trošak se vidi u CSV fajlu. **Česte greške:** tražiti sve tri opcije u tri odvojene poruke — može, ali je sporije; povezane stvari traži zajedno. Suprotna krajnost — 10 funkcionalnosti u jednoj poruci — takođe škodi (teže je proveriti). Zlatna sredina: jedna zaokružena celina po zahtevu.

Rešenje V1.3

Referentni prompt (prvi korak):

Dodaj opciju 'mesečni izveštaj': za izabrani mesec ispiši zbir po kategorijama i ukupan zbir. Predloži sam format ispisa.

pa doterivanje kroz razgovor: Poravnaj iznose udesno i dodaj liniju iznad ukupnog zbira.

Očekivani ishod: čitljiv izveštaj; prošao si bar jedan krug doterivanja. **Poenta:** "predloži sam" je legitimna tehnika — Claude često predloži bolje nego što bi ti specificirao, a ti onda samo koriguješ.

Rešenje V1.4

Referentni prompt: Pokreni `troskovi.py` — puca. Nađi uzrok i ispravi, pa pokreni ponovo da potvrdiš. **Očekivani ishod:** Claude pokrene, pročita traceback, nađe pokvaren red (ili napiše otporniji

kod koji preskače/prijavljuje loše redove), i DOKAŽE ispravku ponovnim pokretanjem. **Česte greške:**

- Prepričavanje greške svojim rečima umesto da ga pustiš da je sam vidi — gubi se tačna informacija. Pusti ga da POKRENE i vidi original.
- Prihvatanje ispravke bez ponovnog pokretanja — uvek traži dokaz.

Bonus pitanje za razmišljanje: da li je bolje da skript pukne na lošem redu ili da ga tiho preskoči? (Odgovor: zavisi — ali "tiho" preskakanje bez ikakve poruke je najgora opcija; traži bar upozorenje.)

Rešenje V1.5

Komanda: `claude -p "koliko redova ima troskovi.csv i koji je najveći pojedinačni trošak?"`

Očekivani ishod: odgovor u terminalu, bez ulaska u razgovor. **Kada koristiti:** brze provere, skriptovi, automatizacija. Za bilo šta gde slede dodatna pitanja — interaktivni režim je bolji.

Rešenje V1.6

Postupak: `claude --resume` → izbor jučerašnje sesije → Dodaj kategoriju 'ostalo': svaki trošak unet bez kategorije automatski je dobija. **Očekivani ishod:** Claude nastavlja sa punim znanjem konteksta — ne moraš mu objašnjavati šta je projekat. **Česta greška:** pokretanje obične nove sesije (`claude`) i čuđenje što "ne zna" o čemu pričaš. Nova sesija = prazno pamćenje (osim CLAUDE.md — Modul 2!).

Rešenje V1.7

Postupak: posle maglovitog zahteva i lošeg rezultata → `/rewind` → izbor tačke pre izmene → precizan zahtev:

Preuredi ispis troškova u tabelu sa kolonama: datum (dd.mm.gggg), opis (max 30 znakova), kategorija, iznos poravnat udesno sa 2 decimale. Zaglavlje odvoji linijom.

Poenta: uporedi šta si dobio od "sredi malo izgled" i od preciznog opisa. Razlika koju vidiš = vrednost preciznog opisivanja.

Rešenje V1.8

Postupak: Shift+Tab dok ne piše acceptEdits → zahtev za malu izmenu → primeti da izmena fajla prolazi bez pitanja, a pokretanje skripta i dalje pita. **Odgovor na pitanje kada je opasan:** folder bez git-a (nema vraćanja), fajlovi koji nisu samo kod (dokumenti, konfiguracije sa lozinkama), rad na tuđem/produkcijskom kodu. Pravilo: acceptEdits tek kad postoji sigurnosna mreža (git) i poverenje u zadatak.

Rešenje V1.9

Kriterijumi uspeha (proveri sam):

- ☐ Postoji nova opcija u meniju koja pravi JSON fajl mesečnog izveštaja
- ☐ JSON sadrži: mesec, zbir po kategorijama, ukupan zbir
- ☐ Koristio si `@troskovi.py` bar jednom u zahtevu

- ☐ Claude je sam pokrenuo i proverio kod bar jednom
- ☐ Proverio si rezultat sa `claude -p "pročitaj izvestaj-<mesec>.json i reci da li je validan JSON i da li se zbir po kategorijama slaže sa ukupnim"`

Referentni prompt za glavni deo:

@troskovi.py Dodaj opciju 'izvoz u JSON': za izabrani mesec napravi fajl izvestaj-GGGG-MM.json sa strukturom {mesec, po_kategorijama: {...}, ukupno}. Pokreni, izvezi jedan mesec i pokaži mi sadržaj fajla.

Česta greška: raditi sve u starom, dugačkom razgovoru — proveru je tražila NOVU sesiju (`/clear`) baš da vidiš koliko toga zavisi od konteksta.

Modul 2 — rešenja

Rešenje V2.1

Referentni prompt:

Napravi početnu stranu ličnog portfolio sajta: index.html sa sekcijama 'O meni', 'Projekti', 'Kontakt', i izmišljenim sadržajem. Čist HTML i CSS.

Očekivani ishod: bilo kakav pristojan sajt. Ovde nema greške — poenta vežbe je u onome što sledi sutra.

Rešenje V2.2

Očekivani ishod: nova stranica koja se verovatno RAZLIKUJE od početne — drugi ton boja, možda engleski tekst, možda inline stilovi umesto zajedničkog CSS-a. Zapisao si bar 2 konkretne razlike. **Ako se slučajno savršeno uklopila:** pogledaj kako — verovatno je Claude pročitao index.html i imitirao ga. To je dobro, ali NIJE garantovano; CLAUDE.md je garancija. (I dalje uradi V2.3.)

Rešenje V2.3

Kompletna primer CLAUDE.md za portfolio projekat:

```
# Portfolio sajt

## Pravila
- Sav tekst na sajtu je na srpskom (latinica).
- Čist HTML/CSS/JS – bez framework-a, biblioteka i CDN linkova.
- Sve stranice koriste zajednički style.css – bez inline stilova i bez <style> blokova u HTML-u.
- Paleta boja (samo ove): pozadina #1a1a2e, tekst #eaeaea, akcenat #e94560, sekundarna #16213e. Definisane kao CSS varijable u :root – nove boje se ne uvode bez dogovora.
- Imena fajlova: mala slova, bez razmaka (kontakt.html, ne Kontakt Strana.html).
- Svaka stranica ima isti header sa navigacijom – pri dodavanju nove stranice dodati link u navigaciju NA SVIM stranicama.
- Sajt se testira otvaranjem index.html u browseru (nema build koraka).
```

Zašto baš ovako: svako pravilo je (1) proverivo, (2) nešto što se NE VIDI očigledno iz koda ili se lako prekrši, (3) kratko. Loš primer pravila: "sajt treba da bude lep" — neproverivo.

Rešenje V2.4

Očekivani ishod: kontakt stranica poštuje jezik, boje, zajednički CSS i navigaciju — bez podsećanja. **Ako nešto nije ispoštovano:** pravilo je verovatno mutno. Primer: "koristi naše boje" (mutno) vs. paleta sa hex vrednostima (jasno). Prepravi pravilo i testiraj ponovo u novoj sesiji — to je normalan ciklus doterivanja CLAUDE.md.

Rešenje V2.5

Postupak za #: ukucaš npr. # dugmad i linkovi uvek imaju hover efekat → Claude pita/pokaže gde upisuje → proveriti da je pravilo završilo u CLAUDE.md. **Za /init:** generisani fajl obično sadrži: opis projekta, strukturu, komande. Kritički pogled: da li je nešto od toga očigledno iz koda (kandidat za brisanje)? Da li fali neka zamka koju samo ti znaš (kandidat za dodavanje)? /init je POČETNA TAČKA, ne gotov proizvod.

Rešenje V2.6

Primer globalnog ~/.claude/CLAUDE.md :

```
# Moja globalna pravila
- Komuniciraj sa mnom na srpskom.
- Commit poruke: kratke, na srpskom, počinju glagolom ("Dodaje...", "Ispravlja...").
- Kad predlažeš rešenje, prvo jedna rečenica ŠTA predlažeš, pa detalji.
```

Provera: u sesiji u portfolio folderu pitaj "koja pravila trenutno važiš?" — treba da nabroji i globalna i projektna. Sukob pravila? Specifičnije (projektno) po pravilu pobeđuje — ali najbolje je ne praviti sukobe.

Rešenje V2.7

Kriterijumi uspeha:

- ☐ Stranica cenovnika je na srpskom (ili tvom jeziku iz pravila)
- ☐ Koristi isključivo boje iz palete i zajednički style.css
- ☐ Navigacija ažurirana na svim stranicama
- ☐ NISI ga nijednom podsetio ni na jedno pravilo

Ako je palo: nađi TAČNO pravilo koje je prekršeno → pročitaj ga očima nekoga ko ne zna projekat → prepisi ga konkretnije → ponovi test u novoj sesiji. Ovaj ciklus (test u čistoj sesiji → rupa → zakrpa) je i profesionalna praksa za CLAUDE.md.

Modul 3 — rešenja

Rešenje V3.1

Referentni prompt (u plan mode):

Napravi web kviz aplikaciju: 10 pitanja opšte kulture, svako sa 4 ponuđena odgovora od kojih je 1 tačan, prikaz jednog pitanja po ekranu, na kraju rezultat (X/10). Čist HTML/CSS/JS, radi otvaranjem index.html.

Šta tražiš u planu pre odobravanja: spisak fajlova (očekuješ index.html, style.css, nešto za logiku i podatke pitanja), redosled koraka, da NEMA nepotrebnih tehnologija (build alati, framework — nisi ih tražio). Ako plan predviđa nešto što ne razumeš — PITAJ pre odobravanja, to je poenta plan mode-a.

Rešenje V3.2

Primer doterivanja plana: plan stigne → odgovoriš npr:

Izmeni plan: 1) rekordi neka čuvaju i datum igranja, 2) dodaj dugme 'obriši sve rekorde' sa potvrdom pre brisanja.

→ stigne doraden plan → tek onda odobriš. **Očekivani ishod:** vidiš da se plan menja pregovaranjem, bez jednog reda napisanog koda. **Česta greška:** odobriti plan "reda radi" pa tražiti izmene POSLE implementacije — time si platio implementaciju pogrešne verzije.

Rešenje V3.3

Referentni prompt:

Razmisli dobro (think hard) pa napravi plan: tajmer 15 sekundi po pitanju (isteklo = netačno), bonus poeni za brzinu odgovora. Obrati pažnju na iverice: promena taba usred pitanja, odgovor u poslednjoj sekundi, šta se dešava sa tajmerom između pitanja.

Očekivani ishod: plan koji EKSPPLICITNO adresira nabrojane iverice (npr. pauziranje tajmera van fokusa ili jasna odluka da se ne pauzira). Uporedi: bez "think hard" i bez nabiranja iverica, plan je tipično površniji baš na ivericama. **Poenta:** nabiranje iverica u zahtevu je TVOJ posao — ti znaš šta te brine; "think hard" daje prostor da se to promisli.

Rešenje V3.4

Referentni postupak: `/loop` pa opis zadatka i intervala (tačna sintaksa: pogledaj `/help`). Bezopasan primer: provera da li svaki .html fajl ima link u navigaciji. **Očekivani ishod:** zadatak se ponovio bar 2 puta; zaustavio si ga bez problema. **Ideje za stvarni život:** nadgledanje da li je build prošao; periodična provera TODO komentara; provera da li su svi testovi zeleni posle serije izmena.

Rešenje V3.5

Referentni prompt (plan mode):

Refaktoriši kviz u module: pitanja.js (samo podaci), kviz.js (logika toka), tajmer.js, rekordi.js (localStorage), prikaz.js (DOM). index.html ih učitava. Ponašanje aplikacije mora ostati IDENTIČNO – ništa se ne menja korisniku. Napravi plan koji uključuje i kako ćemo proveriti da ništa nije puklo.

Šta tražiš u planu: korake koji idu deo-po-deo (ne "sve odjednom"), i korak PROVERE na kraju. **Kriterijum uspeha:** posle refaktorisanja odigrao si celu partiju + rekordi rade + tajmer radi. Refaktorisanje bez provere ponašanja nije završeno.

Rešenje V3.6

Kriterijumi uspeha:

- ☐ Ceo tok kroz plan mode: zahtev → plan → BAR JEDNA tvoja dorada plana → odobrenje → implementacija
- ☐ Režim 2 igrača: naizmenična pitanja ili naizmenične runde (tvoj izbor — ali svesno donet u fazi plana)
- ☐ Uporedni rezultat na kraju + pobednik
- ☐ Lično testirano: partija u režimu 1 igrača I partija u režimu 2 igrača

Tipična zamka: režim 2 igrača otvara pitanja koja plan mora da reši (ista pitanja za oba? šta sa rekordima?) — ako ih plan nije pomenuo, to je trebalo da tražiš u doradi.

Modul 4 — rešenja

Rešenje V4.1

Referentni prompt:

```
Napravi aplikaciju 'prognoza i beleške': SQLite baza beleske.db sa tabelom beleske (id, datum, tekst), i jednostavan interfejs (predloži sam – može Python sa malim web serverom) za unos beleške za dan i pregled beležaka po danima. Ubaci 6 probnih beležaka u poslednjih nedelju dana. Pokreni i pokaži mi kako da otvorim.
```

Očekivani ishod: radi unos i pregled; baza postoji kao fajl.

Rešenje V4.2

Postupak: pitaj Claude-a za tačnu komandu za tvoje okruženje — MCP serveri se instaliraju raznim komandama i menjaju se vremenom, pa je "pitaj alat" pouzdanije od prepisivanja iz kursa. Tipično dobiješ nešto oblika `claude mcp add <ime> -- <komanda koja pokreće server sa putanjom do baze>`. **Provera:** `claude mcp list` prikazuje server kao aktivan; pitanje "koliko beležaka ima u bazi?" dobija odgovor kroz MCP alat (videćeš poziv alata u sesiji), ne kroz pisanje skripta. **Česta greška:** dodati server pa pitati u STAROJ sesiji — po pravilu treba nova/restartovana sesija da server bude vidljiv.

Rešenje V4.3

Očekivano zapažanje: sa MCP-om — jedan poziv alata, direktan odgovor. Bez MCP-a — Claude piše i pokreće skript (više koraka, više odobravanja, sporije), ali TAKOĐE dođe do odgovora. **Zaključak koji treba da ponesesh:** MCP nije jedini način — on je BRŽI i ČISTIJI kanal za stvari koje često radiš. Za jednokratan upit, skript je sasvim OK; za svakodnevni rad sa bazom, MCP se isplati.

Rešenje V4.4

Referentni prompt:

```
Dodaj u aplikaciju prikaz trenutne prognoze za [tvoj grad]: koristi neki besplatan API bez registracije (predloži i obrazloži izbor), prikaži temperaturu i opis vremena na stranici sa beleškama. Obradi i slučaj kad API ne odgovara – aplikacija ne sme da pukne.
```

Očekivani ishod: prognoza na stranici; aplikacija preživi i kad nema interneta (isprobaj!).

Rešenje V4.5

Postupak: instalacija browser MCP-a po uputstvu za tvoje okruženje (pitaj Claude-a šta je dostupno). Zatim:

Otvori aplikaciju u browseru, unesi novu belešku "test iz browsera" za današnji dan kroz formu, i proverí da se pojavila na spisku. Javi šta si tačno uradio i šta si video.

Očekivani ishod: Claude sam klikće/kuca/číta stranicu i vrati izveštaj. **Kriterijum:** beleška zaista postoji u bazi (proveri kroz SQLite MCP — lepa prilika da spojiš oba).

Rešenje V4.6

Nema jednog tačnog rešenja — kriterijum je proces: (1) pogledao si šta postoji, (2) izabrao server sa OBRAZLOŽENJEM šta ti daje, (3) dodao ga i (4) izveo smislen mini-zadatak. Primer: GitHub MCP + "izlistaj otvorene issue-e na repou X i predloži koji je najlakši za početnika". **Bezbednosni check koji je trebalo da uradiš:** da li je server zvaničan/poznat? Šta sve može (samo číta ili i piše)?

Rešenje V4.7

Referentni odgovor na teorijski deo (uporedi svoj): API poziv iz koda je deo APLIKACIJE — izvršava se svaki put kad aplikacija radi, za svakog korisnika (prognoza MORA biti API poziv iz koda, jer korisnici aplikacije nemaju Claude). MCP server je alat za CLAUDE-a tokom RAZVOJA — daje mu oči/ruke ka spoljnim sistemima dok ti pomaže (čítanje baze tokom razvoja je idealno za MCP, jer korisnicima to ne treba, a tebi i Claude-u ubrzava rad). Pravilo palca: **šta treba aplikaciji → kod; šta treba Claude-u da bi ti pomogao → MCP. Praktični deo — kriterijumi:**

- ☐ Claude je kroz browser MCP prošao kroz bar 2 scenarija (unos, pregled)
- ☐ Kroz SQLite MCP potvrdio stanje u bazi posle scenarija
- ☐ Izveštaj sadrži šta radi / šta ne radi / šta je čudno
- ☐ Bar jedan nalaz iz izveštaja si lično proverio (poverenje se proverava!)

Modul 5 — rešenja

Rešenje V5.1

Referentni prompt:

Pošalji Explore subagenta da istraži kako ovaj projekat rukuje greškama (exceptions/error handling): gde je centralna logika, koje tipove grešaka definiše, kako korisnik biblioteke dobija greške. Neka vrati kratak izveštaj sa putanjama ključnih fajlova.

Očekivani ishod: izveštaj od 10–20 redova sa konkretnim putanjama; tvoja sesija nije "otežala" (agent je čitao u svom kontekstu). **Česta greška:** premutan zadatak agentu ("istraži projekat") — agent je NAJBOLJI kad ima usko, konkretno pitanje. Subagent ne vidi vaš razgovor — sve što mu treba mora biti u zadatku.

Rešenje V5.2

Referentni prompt:

Pokreni 3 Explore subagenta PARALELNO: (1) kako se pišu i pokreću testovi u ovom projektu, (2) kako je organizovana konfiguracija, (3) koje spoljne zavisnosti koristi i za šta služi svaka. Kad završe, spoji nalaze u jedan pregled projekta za novog člana tima.

Očekivani ishod: tri agenta rade istovremeno; dobiješ objedinjen pregled. Primeti koliko bi konteksta pojelo da je sve to čitano u tvojoj sesiji.

Rešenje V5.3

Očekivano zapažanje: direktno čitanje = brže za MALA pitanja (nema "režije" pokretanja agenta), ali kontekst ti se vidno napuni; subagent = isplativ kad odgovor zahteva čitanje mnogo fajlova, i kad ti treba samo ZAKLJUČAK a ne sirovi sadržaj. **Tvoje pravilo palca (uporedi):** pitanje o 1–2 poznata fajla → direktno; "kako je X organizovano kroz ceo projekat" → subagent; više nezavisnih pitanja → paralelni subagenti.

Rešenje V5.4

Kompletan primer `.claude/agents/recenzent.md` :

name: recenzent

description: Read-only pregled koda po checklist-i. Koristiti kad se traži recenzija/review koda. Vraća listu nalaza, ništa ne menja.

tools: Read, Grep, Glob

Ti si strog ali pravedan recenzent koda. NIKAD ne menjaš fajlove – samo čitaš i izveštavaš.

Pregledaj kod po ovoj checklist-i:

1. Čitljivost: imena promenljivih/funkcija govore šta rade?
2. Ponavljanje: isti kod na više mesta koji je trebalo izdvojiti?
3. Očigledni bagovi: poređenja, granice petlji, null/undefined, ivice.
4. Rukovanje greškama: šta se dešava kad nešto pođe po zlu?
5. Mrtav kod: funkcije/promenljive koje se nigde ne koriste.

Format izveštaja – za svaki nalaz:

- [putanja:linija] Kratak opis problema → konkretan predlog ispravke
- Ozbiljnost: BAG / RIZIK / STIL

Na kraju: 2 rečenice ukupnog utiska. Ne prijavljuj stilske sitnice koje ne otežavaju razumevanje. Maksimalno 10 najvažnijih nalaza.

Zašto ovako: `tools: Read, Grep, Glob` ga FIZIČKI sprečava da menja (read-only nije molba nego ograničenje); `description` govori KAD se koristi (bitno za automatsko pozivanje); format izveštaja je fiksiran da bi izlaz bio uporediv iz puta u put; limit nalaza sprečava zatrpavanje.

Rešenje V5.5

Postupak: u folderu kviza pozovi recenzenta ("pusti agenta recenzent na ceo projekat"). Očekuj 5–10 nalaza; tipično za vibe-coded kviz: dupliran DOM kod, magic numbers (15 sekundi upisano na 3 mesta), rukovanje praznim imenom igrača. **Kriterijum:** za svaki nalaz si doneo SVOJU odluku (ispraviti/ignorirati + zašto). Recenzent (i AI uopšte) predlaže — ti presuđuješ.

Rešenje V5.6

Kriterijumi uspeha:

- ☐ Tri pitanja su TVOJA i dovoljno uska da agent može konkretno da odgovori
- ☐ Svaki nalaz u izveštaju ima putanje fajlova (bez putanja = preporučavanje, ne analiza)
- ☐ Tvoja procena "šta bi bilo najteže menjati" citira konkretne nalaze (npr. "logika X je raspoređena kroz N fajlova pa svaka izmena dira sve")

Ako izveštaj nema putanje: doradi zadatak agentu — eksplicitno traži putanje. Agent radi tačno ono što piše u zadatku.

Modul 6 — rešenja

Rešenje V6.1

Referentni prompt:

Inicijalizuj git repo. Napravi .gitignore prikladan za [tip projekta] i CLAUDE.md sa osnovnim pravilima projekta (jezik: srpski, [tvoja pravila]). Zatim implementiraj [prva funkcionalnost] i predloži commit – ali mi pokaži poruku pre nego što commit-uješ.

Šta proveravaš u commit poruci: da OPISUJE ŠTA izmena radi (ne "izmene" ili "update"), kratko i konkretno. Loša: "dodao fajlove". Dobra: "Dodaje unos i listanje obaveza sa čuvanjem u JSON".

Rešenje V6.2

Referentni tok: Napravi granu funkcija-X i implementiraj: [opis] → kad završi → /code-review → pročitaj nalaze → Ispravi nalaze 1 i 3, nalaz 2 je namerno tako → commit. **Poenta:** review PRE commit-a znači da u istoriji ostaje čista verzija. **Česta greška:** slepo ispravljanje SVIH nalaza — review alat ne zna tvoje namere; ti presuđuješ koji su nalazi opravdani.

Rešenje V6.3

Referentni prompt: Objavi ovaj repo na mom GitHub nalogu kao javni repo [ime], push-uj main i granu, i otvori Pull Request sa grane na main sa opisom šta funkcionalnost radi i kako je testirana. **Očekivani ishod:** repo na GitHub-u; PR sa smislenim opisom; posle pregleda u browseru si ga merge-ovao (možeš i kroz Claude: "merge-uj PR"). **Preduslov koji se često zaboravi:** gh auth login mora biti urađen pre ovoga.

Rešenje V6.4

Primer .claude/settings.json sa hook-om (Python projekat — provera sintakse posle svake izmene fajla):

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "python -m py_compile \"$CLAUDE_FILE_PATHS\" && echo OK sintaksa"
          }
        ]
      }
    ]
  }
}
```

(Tačna imena događaja i promenljivih proveri u dokumentaciji — `/help` ili traži od Claude-a "podesi mi hook koji posle svake izmene .py fajla proveri sintaksu" i pusti ga da sam napiše `settings.json` — pa PROČITAJ šta je napisao.) **Test:** zatraži izmenu koda → u sesiji vidiš da se hook okinuo. Zatim namerno traži da ubaci sintaksnu grešku — hook treba da je uhvati. **Odgovor na pitanje iz vežbe:** u hook idu pravila koja MORAJU da važe (formatiranje, testovi, zabrana commit-a na main...) — CLAUDE.md je uputstvo koje se čita, hook je kapija koja se ne zaobilazi.

Rešenje V6.5

Kompletna primer `.claude/commands/objavi.md` :

```
---
description: Proveri, commit-uj i push-uj trenutne izmene
---
```

Uradi sledeće, redom, i stani ako bilo koji korak ne prođe:

1. Pokreni proveru koda ako u projektu postoji (testovi/linter); ako ne postoji, preskoči uz napomenu.
2. Pogledaj sve trenutne izmene (git status i git diff).
3. Sastavi kratku commit poruku na srpskom koja opisuje ŠTA izmene rade.
4. Pokaži mi poruku i spisak fajlova, pa sačekaj moju potvrdu.
5. Posle potvrde: commit i push na trenutnu granu.

Argument (opciono): \$ARGUMENTS – ako je prosleđen, koristi ga kao osnovu za commit poruku.

Upotreba: `/objavi` ili `/objavi ispravka tajmera`. **Zašto korak 4 postoji:** komanda koja commit-uje bez pokazivanja = automatizacija bez kontrole; uvek ostavi tačku provere kod nepovratnih koraka.

Rešenje V6.6

Primer skripta (`dnevni-pregled.sh` / analogno za PowerShell):

```
#!/bin/sh
claude -p "Pregledaj projekat: izlistaj sve TODO i FIXME komentare sa
putanjama, grupiši ih po temi i predloži kojim redosledom da se rešavaju
i zašto. Ništa ne menjaj." > dnevni-pregled.txt
```

Očekivani ishod: pokrenut ručno, proizvodi upotrebljiv izveštaj u fajlu. **Ideja dalje:** ovakav skript u CI-ju ili scheduler-u = Claude kao deo automatike, ne samo sagovornik. Napomena: u headless režimu dobro razmisli o dozvolama — kreni od zadataka koji samo ČITAJU.

Rešenje V6.7

Kriterijumi uspeha završnog ispita — pored checklist-e iz zadatka, kvalitativno:

- ☐ Možeš da pokažeš repo nekome i objasniš svaku odluku (zašto plan mode baš tu, zašto taj MCP, šta hook garantuje)
- ☐ CLAUDE.md je takav da bi novi čovek (ili nova sesija) nastavio rad bez tvojih usmenih objašnjenja — TESTIRAJ: nova sesija + zahtev za malu doradu
- ☐ Istorija commit-ova čita se kao priča o nastanku projekta
- ☐ Postoji bar jedan trenutak gde si ODBIO nešto što je Claude predložio, sa razlogom — to je znak da vodiš ti, a ne alat

Ako je sve prošlo: čestitam — prošao si put od "kucam promptove" do "vodim agenta kroz ceo životni ciklus softvera". To je bio cilj kursa.

Kurs je slobodan za deljenje. Pravljen po javno dostupnoj dokumentaciji Claude Code-a (<https://code.claude.com/docs>); komande i nazivi mogu vremenom da se menjaju — kad nešto ne radi kako kurs kaže, pitaj sam alat (`/help`) ili proveriti dokumentaciju.